

Universidad EAFIT
Departamento de Informática y Sistemas

Práctica Final
Internet: Arquitectura y Protocolos

Integrantes:	Jerónimo Rodríguez Restrepo Simón Tovar Tabares
Asignatura:	Internet: Arquitectura y Protocolos
Semestre:	2025-2
Dominio:	https://proyectofinal.ddns.net
Repositorio:	github.com/JeronimoRodriguezRestrepo/Proyecto-Final-Internet

1. Descripción General

Este proyecto implementa un servicio web seguro desplegado en Amazon Web Services (AWS). El sistema está compuesto por un balanceador de carga NGINX con proxy inverso y certificado SSL, dos servidores de aplicación Flask (uno en inglés y otro en español), una base de datos MySQL y un script de estadísticas con envío de correo. Todo el sistema está contenerizado con Docker y es accesible públicamente mediante HTTPS en el dominio proyectofinal.ddns.net.

2. Arquitectura del Sistema

El sistema utiliza cuatro máquinas virtuales EC2 en AWS:

Máquina	Rol	Puertos expuestos
SW1	Balanceador de carga NGINX + SSL	80 (HTTP), 443 (HTTPS)
SW2	Web Server 1 — Formulario en inglés	5000 (interno)
SW3	Web Server 2 — Formulario en español	5000 (interno)
SW4	Base de datos MySQL	3306 (interno)

Flujo de una petición: el usuario accede a <https://proyectofinal.ddns.net>. La petición llega a SW1 donde NGINX la recibe por HTTPS y la redirige mediante round robin a SW2 o SW3. El servidor procesa el formulario y guarda los datos en MySQL en SW4.

3. Configuración del Balanceador de Carga (SW1)

3.1 Instalar Docker

Conectarse a SW1 por EC2 Instance Connect y ejecutar:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y docker.io
sudo systemctl enable docker && sudo systemctl start docker
sudo usermod -aG docker ubuntu
```

3.2 Instalar docker-compose

```
sudo curl -L "https://github.com/docker/compose/releases/latest/download/\
docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

3.3 Obtener certificado SSL con Let's Encrypt

Instalar Certbot y obtener el certificado en modo standalone:

```
sudo apt install -y certbot
sudo certbot certonly --standalone -d proyectofinal.ddns.net
```

Copiar los certificados al proyecto:

```
mkdir -p ~/balanceador/nginx/certs
sudo cp /etc/letsencrypt/live/proyectofinal.ddns.net/fullchain.pem ~/balanceador/nginx/certs/
sudo cp /etc/letsencrypt/live/proyectofinal.ddns.net/privkey.pem ~/balanceador/nginx/certs/
```

3.4 nginx.conf — proxy inverso + round robin

```
events {}

http {
    upstream servidores_web {
        server <IP-PRIVADA-SW2>:5000;
        server <IP-PRIVADA-SW3>:5000;
    }
    server {
        listen 80;
        server_name proyectofinal.ddns.net;
        return 301 https://$host$request_uri;
    }
    server {
        listen 443 ssl;
        ssl_certificate /etc/nginx/certs/fullchain.pem;
        ssl_certificate_key /etc/nginx/certs/privkey.pem;
        location / {
            proxy_pass http://servidores_web;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

3.5 Levantar el balanceador

```
cd ~/balanceador
sudo docker-compose up -d
```

```
sudo docker ps    # debe aparecer 'balanceador' con status Up
```

4. Servidores de Aplicación (SW2 y SW3)

Ambas apps están desarrolladas en Python con Flask. El formulario registra: nombre, comuna (1-10), carrera de interés (Medicina, Ingeniería, Abogacía, Licenciatura) y la fecha automática. SW2 sirve el formulario en inglés y SW3 en español.

4.1 Estructura de archivos

```
~/app_en/    (o ~/app_es/ en SW3)
├─ app.py
├─ Dockerfile
└─ templates/
    └─ index.html
```

4.2 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install flask pymysql
COPY . .
CMD ["python", "app.py"]
```

4.3 Levantar SW2 (inglés)

```
cd ~/app_en
sudo docker build -t app_en .
sudo docker run -d --name eafit_app_en -p 5000:5000 app_en
```

4.4 Levantar SW3 (español)

```
cd ~/app_es
sudo docker build -t app_es .
sudo docker run -d --name eafit_app_es -p 5000:5000 app_es
```

4.5 Verificar

```
sudo docker ps
# Debe mostrar eafit_app_en o eafit_app_es con status Up
```

5. Base de Datos MySQL (SW4)

MySQL 8.0 corre en un contenedor Docker en SW4. Almacena todos los registros de ambos servidores.

5.1 Levantar MySQL

```
cd ~/database
sudo docker-compose up -d
```

5.2 Crear la tabla de registros

```
sudo docker exec eafit_db mysql -ueafit -peafit2025 registros -e "
CREATE TABLE IF NOT EXISTS registros (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  comuna      TINYINT NOT NULL,
  carrera     VARCHAR(50) NOT NULL,
  fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  servidor    VARCHAR(20) DEFAULT NULL
) CHARACTER SET utf8mb4;"
```

5.3 Verificar registros guardados

```
sudo docker exec eafit_db mysql -ueafit -peafit2025 registros \
-e "SELECT * FROM registros;"
```

6. Script de Estadísticas (SW4)

El script stats.py consulta la base de datos, genera tres gráficas con matplotlib (registros por comuna, por carrera y por servidor) y las envía por correo a ialondonoo@eafit.edu.co.

6.1 Instalar dependencias

```
pip install pymysql matplotlib python-dotenv --break-system-packages
```

6.2 Configurar credenciales SMTP

Generar contraseña de aplicación en myaccount.google.com → Seguridad → Verificación en 2 pasos → Contraseñas de aplicación. Luego crear:

```
# ~/stats/.env
SMTP_USER=tu_correo@gmail.com
SMTP_PASS=xxxx xxxx xxxx xxxx
```

6.3 Ejecutar

```
cd ~/stats
python3 stats.py
```

```
# Salida esperada:
# Total registros: N
# Correo enviado a ialondonoo@eafit.edu.co
```

7. Dominio y DNS

El dominio fue registrado en No-IP (noip.com). Se creó el hostname proyectofinal.ddns.net con un registro tipo A. Se asignó una Elastic IP en AWS a SW1 para que la IP no cambie al reiniciar.

1. Registrar cuenta en noip.com
2. Crear hostname: Dynamic DNS → No-IP Hostnames → Create Hostname
3. Asignar Elastic IP en AWS: EC2 → Elastic IPs → Allocate → Associate con SW1
4. Actualizar el hostname en No-IP con la Elastic IP asignada
5. Verificar: nslookup proyectofinal.ddns.net

8. Security Groups en AWS

Máquina	Puerto	Origen	Propósito
SW1	22	0.0.0.0/0	SSH administración
SW1	80	0.0.0.0/0	HTTP → redirige a HTTPS
SW1	443	0.0.0.0/0	HTTPS balanceador
SW2	22	0.0.0.0/0	SSH administración
SW2	5000	0.0.0.0/0	App Flask inglés
SW3	22	0.0.0.0/0	SSH administración
SW3	5000	0.0.0.0/0	App Flask español
SW4	22	0.0.0.0/0	SSH administración
SW4	3306	0.0.0.0/0	MySQL

9. Verificación del Sistema

Prueba	Procedimiento
HTTPS	Abrir https://proyectofinal.ddns.net . Debe mostrar candado SSL válido.
Round robin	Recargar la página varias veces. Debe alternar entre inglés y español.
Registro	Llenar y enviar el formulario. Debe aparecer mensaje de éxito.
Base de datos	Ejecutar SELECT en SW4 y verificar que el registro aparece con el servidor correcto.
Estadísticas	Ejecutar python3 stats.py en SW4. Debe confirmar envío de correo.

10. Repositorio GitHub

github.com/JeronimoRodriguezRestrepo/Proyecto-Final-Internet

Estructura del repositorio:

```
/
├── app_en/
│   ├── templates/
│   │   └── index.html
│   ├── app.py
│   └── Dockerfile
├── app_es/
│   ├── templates/
│   │   └── index.html
│   ├── app.py
│   └── Dockerfile
├── balanceador/
│   ├── nginx/
│   │   └── nginx.conf
│   └── docker-compose.yml
├── database/
│   └── docker-compose.yml
├── stats/
│   └── stats.py
├── .gitignore
├── README.md
└── Rodriguez_Tovar.pdf
```

Nota: el archivo .env con credenciales SMTP está excluido del repositorio mediante .gitignore.